Tarea 7 — Creación y análisis de índices en MySQL

Jhoan Camilo Arango Ortiz · 2º ASIR online

1. Creación del entorno de trabajo

Se ha creado la base de datos **empresa_indices** y dentro de ella una tabla llamada **empleados** con los campos id (clave primaria), nombre, departamento y salario. A continuación se insertaron 10 registros con datos variados repartidos entre los departamentos IT, RRHH, Contabilidad y Marketing.

```
CREATE DATABASE empresa_indices;
USE empresa_indices;

CREATE TABLE empleados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    departamento VARCHAR(50),
    salario DECIMAL(8,2)
);

INSERT INTO empleados (nombre, departamento, salario) VALUES
('Ana Garcia', 'IT', 2500.00),
('Carlos Lopez', 'RRHH', 1800.00),
('Maria Martinez', 'IT', 3200.00),
('Pedro Sanchez', 'Contabilidad', 2100.00),
('Laura Fernandez', 'IT', 2800.00),
('Jorge Ruiz', 'RRHH', 1950.00),
('Sofia Torres', 'Contabilidad', 2300.00),
('Andres Molina', 'IT', 1750.00),
('Elena Castro', 'Marketing', 2200.00),
('Raul Herrera', 'Marketing', 2600.00);
```

2. Análisis sin índices

Antes de crear ningún índice se ejecutó la consulta de filtrado por departamento para obtener los empleados del departamento IT. La consulta devolvió 4 registros correctamente.

```
SELECT * FROM empleados WHERE departamento = 'IT';
```

```
mysql> INSERT INTO empleados (nombre, departamento, salario) VALUES
    -> ('Ana García', 'IT', 2500.00),
    -> ('Carlos López', 'RRHH', 1800.00),
    -> ('María Martínez', 'IT', 3200.00),
    -> ('Pedro Sánchez', 'Contabilidad', 2100.00),
    -> ('Laura Fernández', 'IT', 2800.00),
    -> ('Jorge Ruiz', 'RRHH', 1950.00),
    -> ('Sofía Torres', 'Contabilidad', 2300.00),
    -> ('Andrés Molina', 'IT', 1750.00),
    -> ('Elena Castro', 'Marketing', 2200.00),
    -> ('Raúl Herrera', 'Marketing', 2600.00);
Query OK, 10 rows affected (0.082 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM empleados WHERE departamento = 'IT';
+----+-----------------+--------------+---------+
| id | nombre          | departamento | salario |
+----+-----------------+--------------+---------+
|  1 | Ana García      | IT           | 2500.00 |
|  3 | María Martínez  | IT           | 3200.00 |
|  5 | Laura Fernández | IT           | 2800.00 |
|  8 | Andrés Molina   | IT           | 1750.00 |
+----+-----------------+--------------+---------+
4 rows in set (0.011 sec)

mysql>
```

Resultado de la consulta sin índice: 4 empleados del departamento IT.

A continuación se analizó la consulta con **EXPLAIN**. El resultado mostró un **Table scan** sobre la tabla completa, lo que indica que MySQL recorrió las 10 filas una a una para encontrar las que cumplían el filtro. No se utilizó ningún índice. El coste estimado fue de 1.25 y las filas analizadas 10.

```
EXPLAIN SELECT * FROM empleados WHERE departamento = 'IT';
```

```
mysql> EXPLAIN SELECT * FROM empleados WHERE departamento = 'IT';
+-----------------------------------------------------------------------------
----------------------------------------+
| EXPLAIN
                                         |
+-----------------------------------------------------------------------------
----------------------------------------+
| -> Filter: (empleados.departamento = 'IT')  (cost=1.25 rows=1)
    -> Table scan on empleados  (cost=1.25 rows=10)
 |
+-----------------------------------------------------------------------------
----------------------------------------+
1 row in set (0.012 sec)

mysql>
```

EXPLAIN sin índice: Table scan sobre las 10 filas de la tabla.

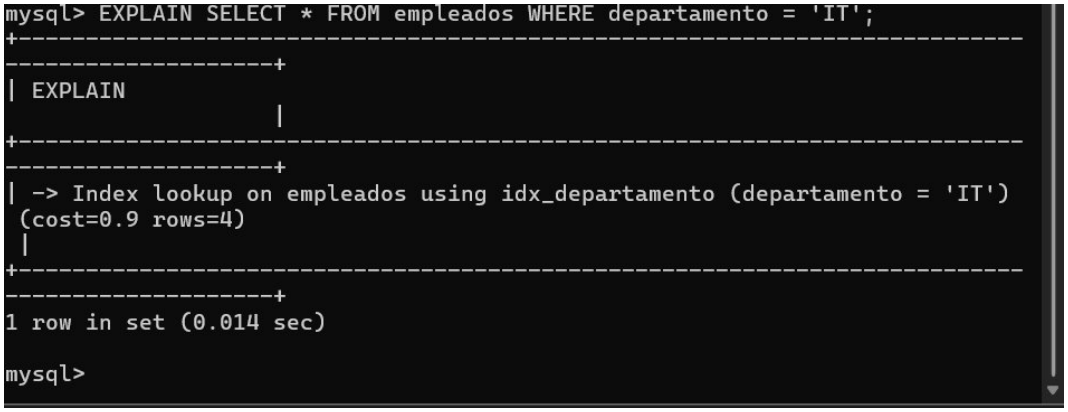
3. Creación de índice sobre departamento

Se creó un índice simple sobre la columna **departamento** con el nombre **idx_departamento**. Este índice permite a MySQL localizar directamente las filas que coinciden con un valor concreto de departamento, sin necesidad de recorrer la tabla completa.

```
CREATE INDEX idx_departamento ON empleados(departamento);
```

4. Análisis con índice

Tras crear el índice se volvió a ejecutar el mismo EXPLAIN. El resultado cambió radicalmente: en lugar de un Table scan, MySQL realizó un **Index lookup** usando **idx_departamento**. El número de filas estimadas bajó de 10 a 4 y el coste se redujo de 1.25 a 0.9. MySQL ya no recorre toda la tabla sino que va directamente a las entradas del índice que corresponden al departamento IT.



```
mysql> EXPLAIN SELECT * FROM empleados WHERE departamento = 'IT';
+-----------------------------------------------------------------------
--------------------+
| EXPLAIN
                    |
+-----------------------------------------------------------------------
--------------------+
| -> Index lookup on empleados using idx_departamento (departamento = 'IT')
 (cost=0.9 rows=4)
 |
+-----------------------------------------------------------------------
--------------------+
1 row in set (0.014 sec)

mysql>
```

EXPLAIN con índice: Index lookup con coste 0.9 sobre 4 filas.

5. Índice compuesto (departamento + salario)

Se creó un índice compuesto sobre las columnas **departamento** y **salario**. Este tipo de índice es útil cuando las consultas filtran habitualmente por ambas columnas a la vez, ya que permite descartar filas usando los dos criterios directamente desde el índice.

```
CREATE INDEX idx_dep_sal ON empleados(departamento, salario);
```

Se ejecutó la consulta con doble filtro y se analizó con EXPLAIN. MySQL usó **idx_departamento** para el filtro de departamento y aplicó el filtro de salario como paso adicional. Con una tabla de solo 10 registros el optimizador considera suficiente el índice simple, pero el coste bajó aún más hasta 0.633 y las filas estimadas a 1.33, lo que demuestra que el índice compuesto aporta información adicional al planificador de consultas. En tablas de gran volumen el índice compuesto sería seleccionado directamente.

```
SELECT * FROM empleados
WHERE departamento = 'IT' AND salario > 2000;
```

```
mysql> SELECT * FROM empleados WHERE departamento = 'IT' AND salario > 2000;
+----+-----------------+--------------+---------+
| id | nombre          | departamento | salario |
+----+-----------------+--------------+---------+
|  1 | Ana García      | IT           | 2500.00 |
|  3 | María Martínez  | IT           | 3200.00 |
|  5 | Laura Fernández | IT           | 2800.00 |
+----+-----------------+--------------+---------+
3 rows in set (0.012 sec)

mysql> EXPLAIN SELECT * FROM empleados WHERE departamento = 'IT' AND salario > 2000;
+-----------------------------------------------------------------------------------------
-------------------------------------------------------------------------------+
| EXPLAIN
                                                                                |
+-----------------------------------------------------------------------------------------
-------------------------------------------------------------------------------+
| -> Filter: (empleados.salario > 2000.00)  (cost=0.633 rows=1.33)
    -> Index lookup on empleados using idx_departamento (departamento = 'IT')  (cost=0.63
3 rows=4)
 |
+-----------------------------------------------------------------------------------------
-------------------------------------------------------------------------------+
1 row in set (0.009 sec)

mysql> |
```

EXPLAIN con índice compuesto: coste 0.633, filtro aplicado sobre 4 filas del índice.

6. Reflexión final

¿Para qué sirven los índices?

Los índices sirven para que MySQL localice los datos sin tener que escanear toda la tabla. Funcionan de forma similar al índice de un libro: en lugar de leer todas las páginas para encontrar un término, el índice apunta directamente a la posición correcta.

¿En qué casos mejoran el rendimiento?

Los índices son especialmente útiles en tablas con un volumen alto de registros y en columnas que se usan frecuentemente en cláusulas WHERE, JOIN u ORDER BY. A pequeña escala la mejora es marginal, pero en tablas con miles o millones de filas la diferencia en tiempo de respuesta es muy significativa.

¿En qué casos pueden perjudicarlo?

Los índices ocupan espacio en disco adicional y tienen un coste de mantenimiento: cada vez que se inserta, modifica o elimina un registro, MySQL tiene que actualizar también todos los índices de esa tabla. En tablas con operaciones de escritura muy frecuentes, tener demasiados índices puede ralentizar esas operaciones más de lo que mejoran las consultas de lectura.